Software Requirements Specification

for

Hostel Management System (HMS)

Part-1 Submission

Prepared by Team 3

Name	SRN
G.V.N. Sreeharsha	PES1UG24CS163
Chennamraju Vaishnavi	PES1UG24CS129
Devansh Gupta	PES1UG24CS145

Date: 07-10-2026

Table of Contents

Revision History
1. Introduction
    1.1 Purpose
    1.2 Intended Audience
    1.3 Product Scope
    1.4 References
2. Overall Description
    2.1 Product Perspective
    2.2 Product Functions
    2.3 User Classes and Characteristics
    2.4 Operating Environment
    2.5 Design and Implementation Constraints
    2.6 Assumptions and Dependencies
3. External Interface Requirements
    3.1 User Interfaces
    3.2 Software Interfaces
    3.3 Communications Interfaces
4. Analysis Models
5. System Features
6. Other Nonfunctional Requirements
    6.1 Performance Requirements
    6.2 Safety Requirements
    6.3 Security Requirements
    6.4 Software Quality Attributes
    6.5 Business Rules
7. Other Requirements
Appendix A: Glossary
Appendix B: Field Layouts
Appendix C: Requirement Traceability Matrix

Revision History

Contributor	Date	Revision / Contribution	Version
G.V.N. Sreeharsha	05-10-2026	Initial SRS baseline: scope, purpose, user classes, functional requirements and non-functional requirements.	1.0
Chennamraju Vaishnavi	07-10-2026	Reviewed requirements from the testing perspective; defined the fee amount (Room Fee) used by REQ-22, added the payment status report to REQ-23 and clarified the hostel search in REQ-7.	1.1
Devansh Gupta	07-10-2026	Reviewed functional and security requirements against the architecture; checked cross-document references and added team number and team member details.	1.2

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Hostel Management System (HMS). HMS is a web based application built using Python and Django for managing student details and the allocation of hostel rooms and beds. This SRS covers the complete system.

1.2 Intended Audience

This document is for:

- Developers - to know what has to be built
- Testers - to write test cases from the requirements
- Faculty evaluator - to review the project
- Hostel office staff - who will be the actual users of the system

Section 2 gives the overall description, Section 3 the interface requirements, Section 4 the use case diagram, Section 5 the functional requirements and Section 6 the non functional requirements.

1.3 Product Scope

Currently the hostel office keeps student details, room allocations and fee payments in paper registers and spreadsheets. Finding a vacant bed takes time, and mistakes such as giving the same bed to two students can happen. Students also cannot check their own allocation or payment status without visiting the office.

HMS solves this by:

- keeping all student, hostel, room and bed details in one place
- showing available beds instantly and blocking double allocation
- recording allocations, transfers and releases with full history
- recording fee payments and generating receipts
- letting students check their allocation and payment status anytime

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
- Django Documentation - https://docs.djangoproject.com
- MySQL 8.0 Reference Manual - https://dev.mysql.com/doc
- OWASP Application Security Verification Standard (ASVS) 4.0
- SE Mini Project Guidelines and Problem Statements, Section C

2. Overall Description

2.1 Product Perspective

HMS is a new, standalone system that replaces the manual hostel registers and spreadsheets. It is not part of any existing system. It has three tiers:

Tier	Technology
Front end	HTML, CSS, Bootstrap (Django templates)
Back end	Python, Django
Database	MySQL

HMS does not connect to any external system in this version.

2.2 Product Functions

- Login and logout for Admin, Warden and Student
- Manage staff user accounts
- Manage students (add, search, update, deactivate)
- Manage hostels, rooms and beds
- Allocate, transfer and release beds
- Record fee payments and generate receipts
- Generate occupancy report
- View own allocation and payment status (Student)

2.3 User Classes and Characteristics

User Class	Description	Usage
Admin	Hostel office head / IT staff. Manages staff accounts, hostels, rooms and beds. Full access.	Daily
Warden	Hostel staff. Registers students, allocates beds, records payments and views reports. Most important user class.	Daily
Student	Views own profile, allocation and payment status only. Mostly uses mobile.	Occasional

2.4 Operating Environment

Component	Requirement
Server OS	Windows 10+ or Ubuntu 20.04+
Runtime	Python 3.11+, Django 5.x
Database	MySQL 8.0
Client	Any modern browser (Chrome, Firefox, Edge) on laptop or mobile

2.5 Design and Implementation Constraints

- Must be developed in Python using Django (as per problem statement)
- MySQL must be used as the database
- Web application only, no mobile app

- Must be completed within the semester by the project team
- All database access through Django ORM or parameterised queries
- Code on GitHub and follows PEP 8

2.6 Assumptions and Dependencies

- Every student has a unique student ID (SRN)
- Each hostel is either Boys or Girls only
- A bed belongs to exactly one room and a room to exactly one hostel
- Fees are collected offline (cash / UPI / bank transfer) and recorded by staff
- Hostel office has a device with internet access
- Depends on third party libraries: mysqlclient and reportlab (PDF receipts)

3. External Interface Requirements

3.1 User Interfaces

- Web interface built with HTML, CSS and Bootstrap
- Role based dashboard shown after login
- Navigation bar with only the options allowed for that role
- Forms show error messages next to the invalid field; mandatory fields marked with *
- Error messages never show stack traces, SQL or file paths
- Pages work on both laptop and mobile screens

3.2 Software Interfaces

Software	Purpose
MySQL 8.0	Stores all data, accessed through Django ORM
Django 5.x	Web framework, authentication, sessions and CSRF protection
reportlab	Generating PDF receipts

3.3 Communications Interfaces

- HTTPS (TLS 1.2 or higher) between browser and server
- MySQL connection between application server and database on a private network

4. Analysis Models

Figure 1 shows the use case diagram with the three actors - Admin, Warden and Student.

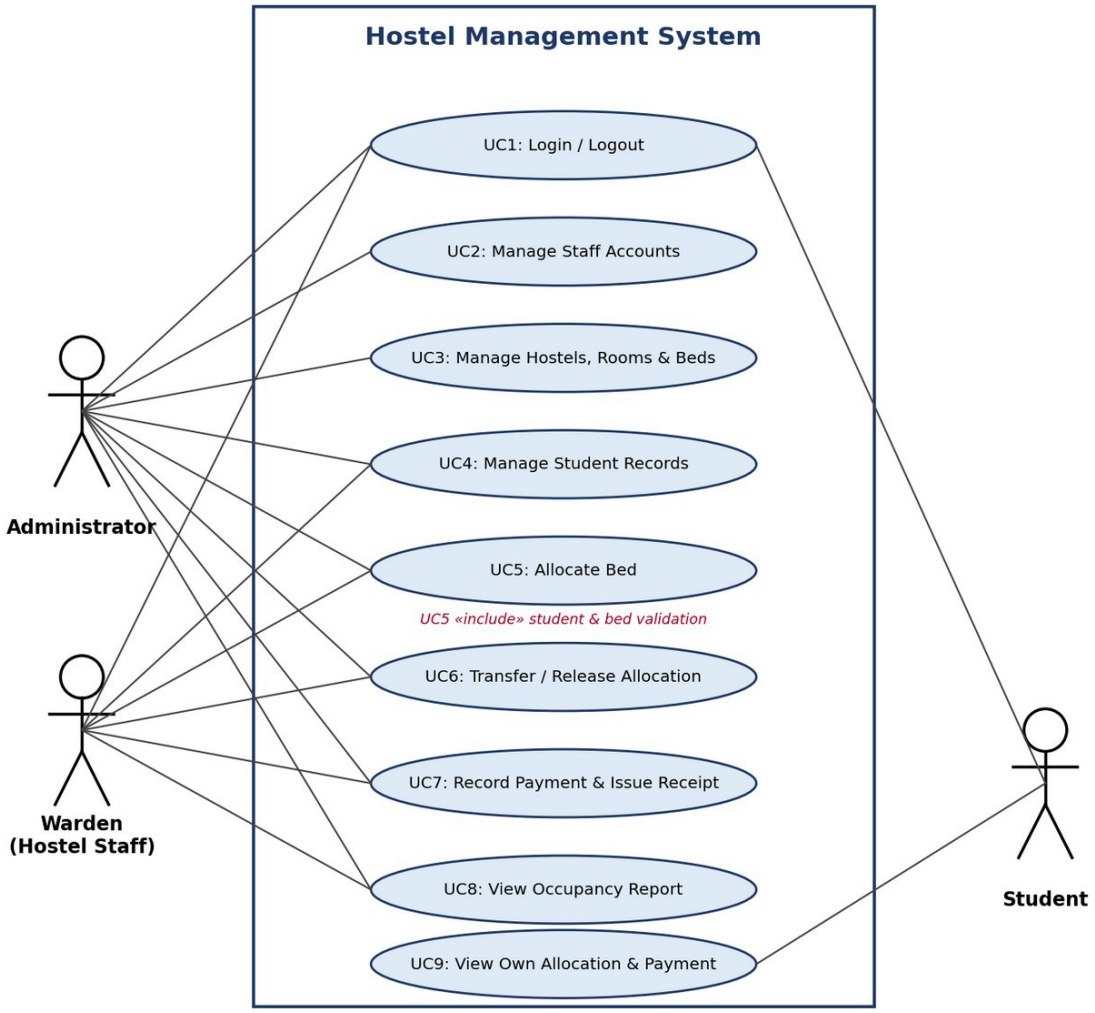


Figure 1: Use case diagram

5. System Features

5.1 User Login and Management

Description and Priority: Lets users login and lets admin manage user accounts. Priority - High.

Stimulus / Response:

User Action (Stimulus)	System Response
User enters correct ID and password	Dashboard for that role is shown
User enters wrong password	"Invalid username or password" shown
Admin submits add user form	User account saved, success message shown

Functional Requirements:

ID	Requirement
REQ-1	The system shall allow Admin, Warden and Student to login using their username and password and shall show the dashboard of their role within 2 seconds.
REQ-2	The system shall show "Invalid username or password" for wrong credentials without telling which one was wrong, and shall lock the account for 15 minutes after 5 consecutive failed attempts.

REQ-3	The system shall end the session when the user clicks Logout or after 15 minutes of inactivity.
REQ-4	The Admin shall be able to create and deactivate user accounts and assign a role (Admin / Warden / Student), with a unique username for each account.

5.2 Student Management

Description and Priority: Lets staff maintain student details needed for hostel admission. Priority - High.

Stimulus / Response:

User Action (Stimulus)	System Response
Staff submits add student form	Student saved, success message shown
Staff submits a Student ID that already exists	"Student ID already registered" shown
Staff searches by ID or name	Matching students listed

Functional Requirements:

ID	Requirement
REQ-5	The Admin or Warden shall be able to register a student by entering all mandatory fields given in Appendix B, and the system shall save the record only if all fields are valid.
REQ-6	The system shall not allow two students with the same Student ID and shall show "Student ID already registered".
REQ-7	The Admin or Warden shall be able to search students by Student ID, name (partial match) or hostel (the hostel of the student's current allocation), with results shown within 2 seconds for up to 5,000 records.
REQ-8	The Admin or Warden shall be able to update a student's contact details, course and year, but not the Student ID.
REQ-9	The system shall deactivate (not delete) a student record only if the student has no active allocation; otherwise it shall refuse and show the allocated bed.

5.3 Hostel, Room and Bed Management

Description and Priority: Lets admin set up hostels, rooms and beds so that allocation can be done. Priority - High.

Stimulus / Response:

User Action (Stimulus)	System Response
Admin adds hostel	Hostel saved and shown in hostel list
Admin adds room with capacity	Room saved and beds created automatically
Staff opens Available Beds	List of free beds shown

Functional Requirements:

ID	Requirement
REQ-10	The Admin shall be able to add and edit a hostel with a unique name and a type (Boys / Girls).
REQ-11	The Admin shall be able to add a room to a hostel with a room number unique within that hostel, floor, room type (single / double / triple) and capacity from 1 to 4.
REQ-12	The system shall automatically create one bed for each unit of room capacity (for example 101-A,

	101-B) with status "Available".
REQ-13	The system shall show available beds filtered by hostel, floor and room type, and shall not show occupied or under-maintenance beds.

5.4 Allocation Management

Description and Priority: Lets staff allocate, transfer and release beds without conflicts. Priority - High.

Stimulus / Response:

User Action (Stimulus)	System Response
Staff selects student and an available bed and confirms	Allocation saved, bed marked Occupied, confirmation shown
Staff selects an already occupied bed	Request rejected with reason shown
Staff releases an allocation	Bed becomes Available, vacate date saved

Functional Requirements:

ID	Requirement
REQ-14	The Admin or Warden shall be able to allocate an available bed to an active student, and the system shall save the allocation date and mark the bed Occupied in one atomic transaction.
REQ-15	The system shall not allow allocation of an occupied bed, or allocation to a student who already has an active allocation, and shall show the reason.
REQ-16	The system shall allow allocation only if the student's gender matches the hostel type.
REQ-17	The Admin or Warden shall be able to transfer a student to another available bed, and the old bed shall be released and the new bed occupied in one atomic transaction.
REQ-18	The Admin or Warden shall be able to release an allocation by entering a vacate date, and the bed shall become "Available".
REQ-19	The system shall keep all past allocations and show a student's allocation history in date order.

5.5 Fee Payment and Receipt

Description and Priority: Records hostel fees paid offline and gives receipts. Priority - Medium.

Stimulus / Response:

User Action (Stimulus)	System Response
Staff enters amount, mode and date for a student	Payment saved and receipt generated
Staff opens student payment status	Paid / Partial / Pending shown

Functional Requirements:

ID	Requirement
REQ-20	The Admin or Warden shall be able to record a payment for a student with amount greater than zero, payment date, mode (Cash / UPI / Bank Transfer) and optional reference number.
REQ-21	The system shall generate a receipt with a unique sequential receipt number for every payment, which can be viewed and downloaded as PDF.

REQ-22	The system shall show the payment status of each student as Paid, Partial or Pending based on the payments recorded against the fee amount for the student's room type (Room Fee, Appendix B).

5.6 Reports and Student View

Description and Priority: Lets staff see occupancy and lets students see their own details. Priority - Medium.

Stimulus / Response:

User Action (Stimulus)	System Response
Staff selects a hostel and clicks Occupancy Report	Total, occupied and available beds shown
Student opens My Allocation	Own bed and payment status shown

Functional Requirements:

ID	Requirement
REQ-23	The system shall generate an occupancy report for each hostel showing total beds, occupied beds, available beds and occupancy %, and a payment status report listing each student's fee amount, amount paid and status (Appendix B); both downloadable as CSV.
REQ-24	The Student shall be able to view only their own profile, current allocation and payment status.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Requirement
NFR-1	At least 95% of page requests shall complete within 2 seconds with up to 50 users at the same time.
NFR-2	Allocating a bed (REQ-14) shall take less than 3 seconds including saving to the database.
NFR-3	The system shall support 5,000 student records and 2,000 beds without exceeding the response time in NFR-1.

6.2 Safety Requirements

ID	Requirement
NFR-4	The database shall be backed up once every day and shall be restorable with at most 24 hours of data loss.
NFR-5	Allocation and payment records shall never be permanently deleted, only updated or marked inactive.

6.3 Security Requirements

Security Objectives:

ID	Requirement
SO-1	Only authorised users can access the system and each user can only do what their role allows.
SO-2	Allocation and payment records can only be created

	or changed through valid operations by authorised staff, and every change can be traced to a user.
SO-3	Personal details of students (contact, guardian and payment details) are kept confidential.

ID	Requirement
SO-4	The system stays usable for staff during office hours and resists simple attacks such as repeated password guessing.

Security Requirements:

ID	Requirement	Objective
NFR-6	Passwords shall be stored hashed using Django PBKDF2, never in plain text, and shall be at least 8 characters with a letter and a digit.	SO-1
NFR-7	The system shall check the user role on every request and block (HTTP 403) pages and actions not allowed for that role.	SO-1
NFR-8	A student shall only be able to view their own data.	SO-3
NFR-9	User session shall expire after 15 minutes of no activity, and session cookies shall be HttpOnly and Secure.	SO-1
NFR-10	All input shall be validated on the server and all forms shall be protected against CSRF and SQL injection.	SO-2
NFR-11	Every login, failed login, create, update, deactivate, allocate, release and payment action shall be logged with user, action, record and date-time, and kept for at least 1 year.	SO-2
NFR-12	All traffic shall use HTTPS (TLS 1.2 or higher), and an account shall be locked for 15 minutes after 5 failed logins.	SO-4

6.4 Software Quality Attributes

Attribute	Requirement
Usability	A new warden can register a student and allocate a bed within 5 minutes using a one page user guide.
Reliability	System available at least 99% of the time during office hours (8 AM - 8 PM) on working days.
Data Integrity	The database itself shall make sure a bed has at most one active allocation and a student has at most one active allocation.
Portability	Runs on both Windows and Linux servers; works on the latest two versions of Chrome, Firefox and Edge.
Maintainability	Code follows PEP 8 and is split into separate Django apps per module; unit tests cover at least 70% of the application logic.

6.5 Business Rules

ID	Rule
BR-1	A bed can have only one active allocation at a time.
BR-2	A student can have only one active allocation at a time.
BR-3	Boys can be allocated only to Boys hostels and girls only to Girls hostels (domain requirement).
BR-4	Only Admin can add or change hostels, rooms, beds and user accounts.
BR-5	A recorded payment cannot be edited or deleted; corrections are made by a new adjusting entry (domain requirement).
BR-6	A student can access only their own data.

7. Other Requirements

- All tables shall use primary and foreign keys with referential integrity (MySQL InnoDB).
- Student data must be handled as per university data rules.
- Only English language is supported in this version. Dates are shown as DD-MM-YYYY and amounts in INR.

Appendix A: Glossary

Term	Meaning
HMS	Hostel Management System
SRN	Student Registration Number
Warden	Hostel staff member who handles day to day hostel work
Allocation	Assigning a student to a bed for a period of time
Active allocation	An allocation with no vacate date
RBAC	Role Based Access Control
ORM	Object Relational Mapping
CSRF	Cross Site Request Forgery
TLS	Transport Layer Security
Soft delete	Marking a record inactive instead of removing it

Appendix B: Field Layouts

Student Registration Fields

Field	Length	Data Type	Description	Mandatory
Student ID (SRN)	13	Alphanumeric	Student registration number	Y
Name	60	Text	Full name	Y
Date of Birth	10	Date	DD-MM-YYYY	Y
Gender	6	Text	Male / Female	Y
Course	30	Text	e.g. B.Tech CSE	Y
Year	1	Numeric	1 to 5	Y
Mobile	10	Numeric	Student mobile number	Y
Email	80	Alphanumeric	Valid email ID	Y
Guardian Name	60	Text	Parent / guardian	Y
Guardian Mobile	10	Numeric	Emergency contact	Y
Address	200	Text	Permanent address	N
Status	10	Text	Active / Inactive	Y

Hostel, Room and Bed Fields

Field	Length	Data Type	Description	Mandatory
Hostel Name	40	Text	Unique hostel name	Y
Hostel Type	5	Text	Boys / Girls	Y
Room Number	6	Alphanumeric	Unique within hostel	Y
Floor	2	Numeric	Floor number	Y
Room Type	10	Text	Single / Double / Triple	Y
Capacity	1	Numeric	1 to 4	Y
Room Fee	10	Numeric	Fee amount in INR for the room type;	Y

			used for payment status	
Bed Number	8	Alphanumeric	Generated, e.g. 101-A	Y

Field	Length	Data Type	Description	Mandatory
Bed Status	15	Text	Available / Occupied / Maintenance	Y

Allocation and Payment Fields

Field	Length	Data Type	Description	Mandatory
Allocation Date	10	Date	Date bed was assigned	Y
Vacate Date	10	Date	Date bed was released	N
Receipt Number	12	Alphanumeric	Unique, sequential	Y
Amount	10	Numeric	Amount paid in INR, greater than 0	Y
Payment Date	10	Date	Date of payment	Y
Payment Mode	15	Text	Cash / UPI / Bank Transfer	Y
Reference Number	30	Alphanumeric	Transaction reference	N

Report Fields

Occupancy Report	Payment Status Report
Hostel Name	Student ID
Total Beds	Student Name
Occupied Beds	Hostel / Room / Bed
Available Beds	Fee Amount
Beds Under Maintenance	Amount Paid
Occupancy %	Payment Status
Generated On	Generated On

Appendix C: Requirement Traceability Matrix

Sl. No	Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test Case ID
1	REQ-1	Login using ID and password, role dashboard	C1,C2	API-A1 4.1	TBD	TC-AUTH-01	Planned
2	REQ-2	Invalid login message and account lockout	C2	API-A1 4.4	TBD	TC-AUTH-02,03	Planned
3	REQ-3	Logout and session timeout	C2	API-A2 3.9	TBD	TC-AUTH-04	Planned
4	REQ-4	Admin creates / deactivates users and assigns roles	C2	API-A3	TBD		Planned
5	REQ-5	Register student with mandatory fields	C3	SD-1 API-S1	TBD	TC-STU-01,02	Planned
6	REQ-6	Reject duplicate Student ID	C3,C9	SD-1 API-S1	TBD	TC-STU-02	Planned
7	REQ-7	Search students	C3,C9	API-S2	TBD	TC-STU-03	Planned
8	REQ-8	Update student details	C3	API-S3	TBD	TC-STU-04	Planned
9	REQ-9	Deactivate student only if no active allocation	C3,C5	API-S4	TBD	TC-STU-05	Planned
10	REQ-10	Admin adds / edits hostel	C4	API-H1	TBD	TC-HST-01	Planned
11	REQ-11	Admin adds room with capacity	C4	API-H2	TBD	TC-HST-01	Planned
12	REQ-12	Auto-create beds for a room	C4,C9	API-H2	TBD	TC-HST-02	Planned
13	REQ-13	Show available beds with filters	C4	API-H3	TBD	TC-HST-02	Planned
14	REQ-14	Allocate available bed to student	C5,C9	SD-2 API-L1	TBD	TC-ALC-01	Planned
15	REQ-15	Block allocation of occupied bed / student with active allocation	C5,C9	SD-2 API-L1	TBD	TC-ALC-02,03	Planned
16	REQ-16	Gender and hostel type must match	C5	SD-2 API-L1	TBD	TC-ALC-04	Planned
17	REQ-17	Transfer	C5,C9	API-L2	TBD	TC-ALC-05	Planned

		student to another bed					
18	REQ-18	Release allocation	C5	API-L3	TBD	TC-ALC-06	Planned
19	REQ-19	Allocation history	C5	API-L4	TBD	TC-ALC-06	Planned
20	REQ-20	Record payment	C6	API-P1	TBD	TC-PAY-01	Planned
21	REQ-21	Generate receipt with unique number	C6	API-P2	TBD	TC-PAY-01	Planned
22	REQ-22	Show payment status	C6	API-P3	TBD	TC-PAY-02	Planned
23	REQ-23	Occupancy report	C7	API-R1	TBD	TC-RPT-01	Planned
24	REQ-24	Student views own data	C2,C3	API-M1	TBD	TC-SEC-02	Planned
25	NFR-1	95% of pages within 2 seconds for 50 users	C1,C9,C10	4.1 indexes	TBD	TC-PERF-01	Planned
26	NFR-2	Allocation within 3 seconds	C5,C9	SD-2 txn	TBD	TC-PERF-02	Planned
27	NFR-3	Support 5,000 students and 2,000 beds	C9,C10	4.1 indexes	TBD	TC-PERF-03	Planned
28	NFR-4	Daily database backup	C10	4.4 backup	TBD	TC-OPS-01	Planned

Sl. No	Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test Case ID
29	NFR-5	No permanent deletion of records	C9	4.1 soft delete	TBD	TC-STU-05	Planned
30	NFR-6	Password hashing and policy	C2	3.9	TBD	TC-SEC-04	Planned
31	NFR-7	Role check on every request	C2	3.9 4.3	TBD	TC-SEC-01	Planned
32	NFR-8	Student sees only own data	C2,C3	API-M1 3.9	TBD	TC-SEC-02	Planned
33	NFR-9	Session expiry and secure cookies	C2	API-A2 3.9	TBD	TC-AUTH-04,SEC-06	Planned
34	NFR-10	Input validation, CSRF and SQL injection protection	C1,C9	3.9 4.4	TBD	TC-SEC-03,05	Planned
35	NFR-11	Audit logging	C8	4.4	TBD	TC-SEC-07	Planned
36	NFR-12	HTTPS and brute-force lockout	C2,C1	API-A1 3.9	TBD	TC-AUTH-03,SEC-06	Planned